

Atividade Prática 1 — Classificador Bayesiano

Etapa 3 — Implementação do Classificador Naive Bayes em SQL

Autor	Marcus Vinícius Santos de Almeida
Disciplina	Mineração de Dados
Data de entrega	04/09/2026
Banco de dados	SQLite — arquivo dados/classificador.db
Artefatos	sql/classificador_naive_bayes.sql · sql/rodar_classificador.py
Entrada de treino	dados/etapa2_dados_treinamento.csv → tabela dados_treinamento

Este relatório explica a **arquitetura** do código para apoiar a defesa oral — não repete o SQL linha a linha (isso está nos comentários do próprio arquivo). A lógica SQL não mudou desde a versão anterior deste relatório; o que mudou são os dados de entrada (Etapa 2, decisão "Naive Bayes puro" — ver CLAUDE.md), então os números de exemplo abaixo foram recalculados sobre a massa atual.

1. Por que SQLite

- **Roda sem servidor.** É um único arquivo (`classificador.db`); não há processo a subir, porta a abrir nem usuário/senha. Basta a biblioteca padrão do Python (`import sqlite3`).
- **Fácil de testar no ambiente.** O script de teste cria o banco do zero, importa o CSV, roda o classificador e confere o resultado em segundos — sem dependências externas.
- **SQL padrão suficiente.** O classificador usa apenas `CREATE VIEW`, `JOIN`, `GROUP BY`, CTEs (`WITH`), `UNION ALL`, `CASE` e as funções `LN/EXP` — tudo disponível no SQLite 3.45. Nada aqui depende de recurso exclusivo de um SGBD específico; portar para PostgreSQL exigiria mudanças mínimas.

Custo assumido: SQLite tem tipagem fraca e não tem `PIVOT` nativo — o "unpivot" é feito à mão com `UNION ALL` (Seção 3.2), o que na verdade deixa a lógica mais explícita.

2. Fluxo de dados (do CSV ao veredito)

```
CSV da Etapa 2
| importa (rodar_classificador.py)
v
dados_treinamento ..... 150 modulos, formato largo
|
+--> (a) priors ..... P(SIM), P(NAO)
|
+--> (b) treino_longo .... 900 linhas, formato longo
      |
      v
      (c) verossimilhanças P(categoria | classe) + Laplace
          |
          | caso_teste | (6 linhas: o modulo novo a classificar)
          v           |
          (d) score_log ..... ln P(classe) + soma de ln P(cat|classe)
              |
              v
              (e) classificar_modulo P(SIM)%, P(NAO)%, recomendacao
```

3. As views, uma a uma

3.1 priors — probabilidade a priori P(classe)

Conta quantos módulos de treino são `SIM` e quantos são `NAO` e divide pelo total. É o palpite inicial, antes de olhar qualquer feature.

Resultado com a massa da Etapa 2: $P(\text{NAO}) = 98/150 = 0,653$ e $P(\text{SIM}) = 52/150 = 0,347$. A proporção de classes é a mesma da versão anterior do dataset — o gerador fixa `PROPORCAO_DEFEITO = 0,35` via seleção top-k, independentemente de como as 6 features são sorteadas (Etapa 2, §1.4).

3.2 treino_longo — "unpivot" das 6 categorias

`dados_treinamento` tem uma coluna de categoria por feature (`complexidade_cat`, `loc_cat`, ...). Para calcular a verossimilhança de todas as features com **uma** consulta em vez de seis quase iguais, empilhamos os seis recortes com `UNION ALL`, chegando ao formato longo:

`(modulo_id, defeito, feature, categoria)` — 150 módulos × 6 features = **900 linhas**.

O nome curto de cada feature (`complexidade`, `loc`, `n_autores`, `churn`, `n_imports`, `cobertura`) passa a ser a chave usada no resto do pipeline e também nos casos de teste.

3.3 verossimilhanças — P(categoria | classe) com suavização de Laplace

Para cada combinação (feature, categoria, classe), a probabilidade condicional é

$P(\text{feature} = \text{categoria} \mid \text{classe}) = (\text{contagem} + 1) / (\text{total_da_classe} + k)$, com $k = 3$ (as três categorias possíveis: `baixo`, `medio`, `alto`), igual para as 6 features.

Por que o "+1" e o "+k" (Laplace): sem eles, se uma categoria nunca aparece junto com uma classe no treino, a contagem é 0 e a probabilidade condicional é 0. Como o Naive Bayes combina as features multiplicando (ou, aqui, somando os logs), **um único zero anula a classe inteira** — o modelo fica "certo demais" por causa de uma célula vazia que é só falta de dados. O Laplace adiciona meia folga a cada categoria: nenhuma probabilidade é exatamente 0 nem exatamente 1, e o efeito sobre as categorias bem povoadas é desprezível. Como bônus, as três probabilidades de cada (feature, classe) passam a **somar exatamente 1**.

A view gera primeiro **todas** as $6 \times 3 \times 2 = 36$ combinações possíveis (CROSS JOIN de features, categorias e classes) e só então faz `LEFT JOIN` com as contagens observadas — assim as combinações ausentes no treino entram com contagem 0 e recebem o Laplace.

Exemplo concreto: `complexidade = alto` na classe `SIM` aparece 13 vezes; $P = (13 + 1) / (52 + 3) = 14/55 \approx 0,255$.

3.4 score_log — somar logaritmos em vez de multiplicar

Para cada classe, o Naive Bayes calcula

`score(classe) = P(classe) × P(cat1|classe) × ... × P(cat6|classe)`.

Por que log: cada fator é um número menor que 1 (aqui entre ~0,02 e ~0,7). Multiplicar 7 números pequenos dá um valor minúsculo; com muitas features isso chega a **underflow** — o `float` vira 0 e a informação some. Tomando logaritmo, o produto vira **soma**:

`log score(classe) = ln P(classe) + Σi ln P(cati | classe)`

Somar sete números da ordem de -1 a -4 é numericamente seguro. E como `ln` é crescente, a classe de maior log-score é a de maior probabilidade.

A view junta as 6 linhas do caso (`caso_teste`) com as verossimilhanças das **duas** classes e agrupa por (caso, classe), produzindo 2 linhas por caso. A coluna `n_features` deve dar 6 — se vier menor, alguma categoria do caso não casou (nome de feature errado ou categoria fora de `baixo/medio/alto`).

3.5 classificar_modulo — normalização e recomendação

Converte os dois log-scores de volta para probabilidade entre 0 e 100 %:

`P(SIM) = exp(log_sim) / ( exp(log_sim) + exp(log_nao) )`

O denominador comum $P(\text{caso})$ desaparece nessa divisão; reexponenciar e dividir pela soma reintroduz a normalização. Antes do `exp()` subtraímos `log_max = max(log_sim, log_nao)` dos dois expoentes — isso **não muda o resultado** (o fator sai igual no numerador e no denominador), só evita passar argumentos muito negativos ao `exp()`. É a forma numericamente correta da conta.

A recomendação textual sai de um CASE:

Condição	Recomendação
<code>P(SIM) > 50 %</code>	ALTO RISCO — recomenda-se revisão de código e testes adicionais
caso contrário	BAIXO RISCO — pode seguir o fluxo normal

4. Funções LN e EXP no SQLite

O SQLite ≥ 3.35 traz `ln()` e `exp()` **nativas** quando compilado com `SQLITE_ENABLE_MATH_FUNCTIONS`. O `sqlite3` do Python usado aqui (SQLite **3.45.1**) já as inclui — confirmado com `SELECT ln(1), exp(0)`.

Mesmo assim, por portabilidade, `rodar_classificador.py` testa essas funções ao abrir a conexão e, **se falharem**, registra equivalentes em Python via `connection.create_function("ln", 1, math.log)` e `("exp", 1, math.exp)`. O mesmo `.sql` roda então em qualquer build, sem alteração.

5. Teste de ponta a ponta

`sql/rodar_classificador.py` recria o banco, roda o SQL e classifica **dois casos de exemplo mínimos** — apenas um *smoke test* para provar que o pipeline funciona. Os 5+ casos formais e a análise crítica são da Etapa 4.

Caso	Perfil (6 categorias)	P(SIM)	P(NAO)	Recomendação
<code>exemplo_alto_risco</code>	complexidade alto, loc baixo, autores alto, churn alto, imports alto, cobertura baixo	96,64 %	3,36 %	ALTO RISCO
<code>exemplo_baixo_risco</code>	complexidade baixo, loc medio, autores baixo, churn baixo, imports baixo, cobertura alto	4,28 %	95,72 %	BAIXO RISCO

Verificações automáticas do script (falha $\Rightarrow$ código de saída 1):

- as duas probabilidades **somam 100,00 %** nos dois casos;
- **6/6 features** casaram com a tabela de verossimilhanças;
- o veredito bate com a intuição do domínio: arquivo pequeno, denso, muito mexido e mal coberto $\rightarrow$ alto risco; arquivo médio, simples, estável e bem coberto $\rightarrow$ baixo risco.

Log-scores obtidos (mostram o log em ação, longe de qualquer underflow): `exemplo_alto_risco` $\rightarrow$ ln-score SIM $-8,45$ vs. NAO $-11,81$; `exemplo_baixo_risco` $\rightarrow$ ln-score SIM $-9,78$ vs. NAO $-6,67$.

6. Como executar

```
python3 sql/rodar_classificador.py
```

O script é idempotente: recria `dados_treinamento` a partir do CSV e derruba/cria todas as views (`DROP ... IF EXISTS`), então pode ser rodado quantas vezes for preciso. Para inspecionar o modelo manualmente: `sqlite3 dados/classificador.db "SELECT * FROM verossimilhanças;"`.

Nota sobre o uso de IA

O SQL e o runner foram escritos em diálogo com uma IA generativa; cada view foi revisada e testada (contagens conferidas à mão, soma das verossimilhanças = 1, probabilidades finais somando 100 %). Todo o código é comentado bloco a bloco e é defensável oralmente.